CHIANGMAI AIR QUALITY ALERT SYSTEM: END-TO-END MLOPS PIPELINE FOR PM2.5 FORECASTING

by

Supanut Kompayak (st126055)

Shuvam Shrestha (st125975)

A Project Proposal Submitted in Partial Fulfillment of the Requirements for the Course AT82.9002 – Selected Topic: Data Engineering and MLOps

Instructor: Dr. Chantri Polprasert

Course: AT82.9002 Selected Topic: Data Engineering and MLOps

Semester: January 2026

Asian Institute of Technology
School of Engineering and Technology
Thailand

March 2026

ABSTRACT

Chiang Mai, Thailand, consistently ranks among the most severely air-polluted cities in the world during its annual haze season from January to April. Fine particulate matter (PM2.5) regularly exceeds the World Health Organization's 24-hour guideline of 15 μg/m^3 by a factor of three or more, posing significant respiratory and cardiovascular health risks to residents, tourists, and vulnerable populations. Despite the severity and predictability of these pollution episodes, existing public-facing tools provide only real-time monitoring with no forward-looking predictive capability.

This proposal presents the ChiangMai Air Quality Alert System (CAAS), a production-ready end-to-end MLOps pipeline that forecasts PM2.5 concentration levels 1, 3, and 7 days in advance and automatically triggers health alerts when projected concentrations exceed the 50 μg/m^3 hazardous threshold. The system integrates three complementary data sources: historical and real-time PM2.5 observations from Thailand's Pollution Control Department (PCD) monitoring network, meteorological variables from the Open-Meteo API, and fire hotspot detections from NASA FIRMS. A 15-year dataset spanning 2011 to 2025 (approximately 5,475 daily observations) forms the training foundation.

The proposed methodology employs a two-model comparative strategy. An XGBoost model, augmented by 44 engineered temporal and contextual features, serves as the primary baseline and is expected to perform strongly across all horizons given its well-documented efficiency on moderate-sized tabular data. A Long Short-Term Memory (LSTM) network with a 30-day input window is deployed as a comparative sequential model to empirically test whether explicit sequence modeling provides additional forecasting benefit, particularly at longer horizons (t+3, t+7). The selection of the production model will be determined by empirical evaluation rather than assumed in advance.

The MLOps infrastructure is designed for reliability and maintainability, featuring automated daily batch ingestion orchestrated by GitHub Actions, AWS cloud deployment (S3, Lambda, EC2), experiment tracking via MLflow, data drift monitoring via Evidently AI, and automated retraining triggers. The estimated deployment cost is less than $8 per month in cloud resources.

Contents

ABSTRACT **i**

LIST OF ABBREVIATIONS **vi**

1 INTRODUCTION **1**

1.1 Background of the Study 1
1.2 Statement of the Problem 1
1.3 Objectives of the Study 2
1.4 Organization of the Report 3

2 BACKGROUND AND RELATED WORK **4**

2.1 PM2.5 Air Pollution and Public Health 4
2.2 Data-Driven Approaches for Air Quality Forecasting 4
2.3 Tabular Models versus Sequential Models for Time Series 5
2.4 MLOps: Principles and Tools 5
2.5 Related Systems and Novelty 6

3 PROPOSED METHODOLOGY **7**

3.1 System Overview 7
3.2 Data Pipeline Design 7
3.2.1 Data Sources 7
3.2.2 Ingestion Method 8
3.2.3 Transformation Pipeline 8
3.3 Feature Engineering 8
3.3.1 Temporal Features 9
3.3.2 Contextual Environmental Features 10
3.3.3 Target Variables 10
3.4 Model Training and Evaluation 10
3.4.1 Model Selection Rationale 10
3.4.2 Chronological Data Split 12

4 SYSTEM ARCHITECTURE AND PIPELINE DESIGN **13**

4.1 Architecture Overview 13

4.2 Storage Layers 13
4.3 MLOps Lifecycle 14
4.3.1 Experiment Tracking 14
4.3.2 Model Versioning and Promotion 14
4.3.3 Deployment Strategy 15
4.4 Monitoring and Automated Retraining 18
4.4.1 Data Drift Monitoring 18
4.4.2 Model Performance Monitoring 18
4.5 Infrastructure and Cost Estimate 18
4.6 Data Quality Management 19
4.6.1 Missing Value Handling Strategy 19
4.6.2 Schema Validation Rules 19
4.6.3 Pipeline Health Monitoring 20

5 EXPERIMENTAL PLAN AND EXPECTED CONTRIBUTIONS 21
5.1 Dataset Characteristics 21
5.2 Baseline Methods for Comparison 21
5.3 Evaluation Metrics 22
5.4 Experimental Setup 22
5.5 Expected Contributions 22
5.6 Feasibility and Tool Selection 23
5.7 Team Roles and Responsibilities 24
5.8 Known Limitations and Risk Mitigation 24
5.9 Project Timeline 26

REFERENCES 26

List of Tables

1.1 Problem framing summary 2

3.1 Summary of integrated data sources 7
3.2 Temporal autocorrelation and calendar features 9
3.3 Contextual meteorological and fire activity features 10
3.4 Target variables for regression and classification tasks 10
3.5 Chronological training, validation, and test split 12

4.1 Estimated monthly AWS cloud infrastructure cost 18
4.2 Missing value handling by data source 19

5.1 Evaluation metrics by task type and priority 22
5.2 Team member roles and task ownership 24
5.3 Known limitations and risk mitigation strategies 25
5.4 Project milestones and target dates 26

List of Figures

3.1 High-level overview of the ChiangMai Air Quality Alert System (CAAS). 7

4.1 AWS cloud architecture for the ChiangMai Air Quality Alert System (CAAS). The pipeline spans three external data sources (air4thai API, Open-Meteo API, NASA FIRMS API), an AWS Lambda ingestion layer triggered daily at 06:00 UTC+7 via GitHub Actions, four S3 storage tiers (Raw, Processed, Feature Store, Model Artifacts), and a compute layer comprising MLflow Tracking Server, FastAPI Inference Server, Streamlit Dashboard, and Evidently AI drift monitoring. Dashed arrows indicate orchestration triggers and automated retraining signals. . . 13

4.2 MLflow model promotion and lifecycle flow. Newly trained models are registered in MLflow and enter the Staging stage, where they undergo schema validation (44-feature check), performance validation ($R^2 \geq 0.70$), and regression tests against the current Production model. Failed validation keeps the model in Staging with an alert logged. On success, the model is promoted to Production with zero-downtime FastAPI hot-reload; the previous Production model is archived for rollback. Evidently AI drift detection and a daily scheduled pipeline serve as the primary retraining triggers. 15

4.3 Daily MLOps pipeline execution flow triggered at 06:00 UTC+7 via GitHub Actions. The pipeline comprises seven stages: (1) Data Ingestion via AWS Lambda fetching from air4thai, Open-Meteo, and NASA FIRMS APIs into S3 Raw; (2) Schema Validation with a failure branch that sends an alert and stops the pipeline; (3) Data Cleaning including duplicate removal, forward-fill for missing values, and outlier clipping; (4) Feature Engineering producing 44 features (lag, rolling statistics, cyclical encodings, seasonal flags, fire hotspots) stored to S3 Feature Store; (5) Model Inference generating PM2.5 regression values and alert-level classifications for t+1, t+3, and t+7 horizons; (6) Store Predictions to S3 and update the FastAPI cache; (7) Monitoring Check using Evidently AI PSI drift detection, flagging for retraining if PSI > 0.2. 16

4.4 End-to-end PM2.5 forecast request flow for the CAAS system. 17

LIST OF ABBREVIATIONS

API	Application Programming Interface
AQI	Air Quality Index
AWS	Amazon Web Services
CAAS	ChiangMai Air Quality Alert System
CI/CD	Continuous Integration / Continuous Deployment
EC2	Elastic Compute Cloud (AWS)
FIRMS	Fire Information for Resource Management System
FRP	Fire Radiative Power
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MLOps	Machine Learning Operations
PCD	Pollution Control Department (Thailand)
PM2.5	Particulate Matter with diameter $\leq$2.5 µm
PSI	Population Stability Index
REST	Representational State Transfer
RMSE	Root Mean Squared Error
S3	Amazon Simple Storage Service
SHAP	SHapley Additive exPlanations
WHO	World Health Organization

CHAPTER 1
INTRODUCTION

1.1 Background of the Study

Air quality degradation from fine particulate matter (PM2.5) represents one of the most pressing environmental health challenges in Southeast Asia. PM2.5 particles, with aerodynamic diameters of 2.5 micrometers or less, penetrate deep into the respiratory system and are associated with increased incidence of respiratory disease, cardiovascular disease, and premature mortality [1]. The World Health Organization's revised 2021 air quality guidelines set the 24-hour mean PM2.5 threshold at 15 $\mu g/m^3$—a standard that Chiang Mai regularly exceeds by a factor of three to six during its haze season.

Chiang Mai's air quality crisis is driven by a convergence of factors: agricultural burning in the lowlands, wildfires in the surrounding highland forests, transboundary smoke from neighbouring countries (notably Myanmar and Laos), and the city's basin-shaped topography, which limits atmospheric dispersion and traps pollutants near the ground. These conditions create highly predictable annual pollution cycles, with peak concentrations typically occurring between January and April, punctuated by acute events triggered by concentrated fire activity.

Despite the predictability of these patterns, the public information infrastructure remains reactive rather than proactive. Thailand's PCD provides real-time AQI data through the air4thai platform, but no operational system exists to forecast upcoming pollution levels or automatically alert residents and health authorities before dangerous thresholds are breached. The resulting gap between available observational data and actionable early warning represents both a significant public health failure and a data engineering opportunity with real societal impact.

1.2 Statement of the Problem

The core problem addressed by this project is the absence of a predictive, automated early warning system for PM2.5 air quality events in Chiang Mai. Current tools inform the public about pollution that has already occurred; they cannot provide advance notice of conditions likely to develop over the coming one to seven days.

Specifically, the following capability gaps exist in the current landscape:

1. No operational system predicts PM2.5 concentration levels 1, 3, or 7 days in advance for Chiang Mai or Northern Thailand more broadly.

2. No automated mechanism exists to trigger health alerts before PM2.5 reaches hazardous thresholds, denying residents and public health agencies the lead time needed for protective action.
3. No self-updating system adapts to the shifting pollution patterns driven by changing agricultural practices, increasing wildfire frequency, and long-term climate trends.

Table 1.1: Problem framing summary

Dimension	**Description**
Who is affected	Residents, tourists, public health agencies, city planners
Core problem	No predictive early warning system exists—only real-time monitoring
Impact	Preventable health exposure due to lack of advance notice
Gap	Existing tools show current conditions; none forecast what is coming
Proposed solution	End-to-end MLOps pipeline: daily forecast (t+1/3/7) + automated alert + drift-aware retraining

1.3 Objectives of the Study

The primary objective of this project is to design and implement a production-ready MLOps pipeline providing accurate PM2.5 forecasts and automated health alerts for Chiang Mai. The specific objectives are:

1. To design and implement a multi-source data ingestion pipeline that automatically collects PM2.5, weather, and fire hotspot data on a daily basis.
2. To engineer a feature set capturing temporal autocorrelation, seasonal patterns, meteorological influences, and regional fire activity signals.
3. To train and empirically compare XGBoost and LSTM models for PM2.5 regression and alert classification across multiple forecast horizons (t+1, t+3, t+7).
4. To deploy the forecasting system as a REST API and Streamlit dashboard accessible to end users.
5. To implement a monitoring and automated retraining system detecting data drift and model performance degradation using Evidently AI and MLflow.

6. To demonstrate a cost-effective cloud deployment within AWS Learner Lab constraints ($2–8/month target).

1.4 Organization of the Report

Chapter 2 reviews relevant background literature on PM2.5 forecasting, time series modeling, and MLOps principles. Chapter 3 describes the proposed methodology including data pipeline design, feature engineering, and model selection rationale. Chapter 4 details the system architecture and full MLOps lifecycle. Chapter 5 presents the experimental design, evaluation plan, expected contributions, and project timeline.

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 PM2.5 Air Pollution and Public Health

Fine particulate matter (PM2.5) is among the most comprehensively studied environmental health hazards globally. Long-term epidemiological evidence demonstrates statistically robust associations between PM2.5 exposure and respiratory disease, ischemic heart disease, stroke, and lung cancer [1]. The WHO 2021 Global Air Quality Guidelines tightened the 24-hour PM2.5 guideline from 25 μg/m^3 to 15 μg/m^3, reflecting accumulated evidence of health impacts at lower concentrations than previously understood.

Northern Thailand's haze problem is characterised by sharp seasonal spikes driven primarily by biomass burning. Unlike urban vehicular pollution, which tends to be geographically diffuse and temporally consistent, fire-driven PM2.5 episodes are spatially concentrated and temporally clustered. This structure makes them, in principle, predictable from fire activity and meteorological precursors—a property that motivates the data-driven forecasting approach in this project.

2.2 Data-Driven Approaches for Air Quality Forecasting

Traditional approaches to air quality forecasting rely on physics-based chemical transport models (CTMs) such as WRF-Chem and CMAQ, which simulate atmospheric dynamics and chemistry explicitly [2]. While physically interpretable, these models are computationally intensive, require detailed emission inventories, and are difficult to adapt to local conditions without domain expertise.

Data-driven machine learning approaches have increasingly demonstrated competitive performance for short-range air quality forecasting, particularly when rich observational histories are available. Gradient boosting methods such as XGBoost and LightGBM perform strongly on tabular time series when temporal lag features are included, effectively encoding the autoregressive structure of pollution time series into a flat feature representation [4]. Studies in regional PM2.5 forecasting in China, India, and Southeast Asia report R^2 values in the range of 0.70–0.88 for next-day prediction using similar feature engineering approaches [3].

2.3 Tabular Models versus Sequential Models for Time Series

The relative merits of gradient boosting methods and deep sequential models (LSTM, Transformer) for tabular time series prediction have been an active area of benchmarking research. The M4 and M5 forecasting competitions demonstrated that tree-based and statistical ensemble methods consistently match or outperform deep learning approaches across a wide range of time series domains [6].

Shwartz-Ziv and Armon [7] conducted a systematic evaluation demonstrating that gradient boosting methods (XGBoost, CatBoost) generally outperform deep learning on tabular datasets with moderate sample sizes, particularly when domain-relevant features are explicitly engineered. Their findings are directly relevant to the dataset considered in this project: approximately 5,475 daily observations is a moderate-size tabular dataset by machine learning standards.

Hochreiter and Schmidhuber's Long Short-Term Memory architecture [5] was designed specifically to capture long-range temporal dependencies without the vanishing gradient problem of standard RNNs. LSTM has demonstrated advantages over tabular models in domains where the *sequence itself* contains information beyond what fixed-window lag features can encode—for example, in multi-step forecasting tasks where the future trajectory depends on complex interactions across long lookback periods.

Implication for this project: Given the dataset size and the fact that the feature engineering strategy already encodes autocorrelation explicitly through lag and rolling features, XGBoost is expected to perform well across all forecast horizons. LSTM is included as a comparative model to empirically test whether sequential modeling provides additional benefit—particularly at t+3 and t+7 horizons where fixed-lag features may capture less of the relevant dynamics. The production model selection will be determined by empirical evidence, not assumed in advance.

2.4 MLOps: Principles and Tools

MLOps is the practice of applying DevOps principles to machine learning systems to enable reliable, scalable, and maintainable deployment of ML models in production. Sculley et al. [8] identified the pervasive challenge of "hidden technical debt" in ML systems, noting that the model training code typically represents only a small fraction of total system complexity, with data pipelines, serving infrastructure, and monitoring comprising the majority of the codebase and operational burden.

Key operational challenges in production ML systems include: data distribution shift (data drift) causing model predictions to degrade over time, concept drift where the statistical relationship between inputs and targets changes, and the need for reproducible experiment tracking to enable model comparison and rollback [8].

This project addresses these challenges through a combination of MLflow for experiment tracking and model registry management [9], Evidently AI for automated drift detection, and GitHub Actions for CI/CD orchestration. The combination of these tools provides a lightweight but complete MLOps stack appropriate for the project scope and budget constraints.

2.5 Related Systems and Novelty

Thailand's PCD air4thai platform [10] provides real-time AQI data from approximately 90 monitoring stations nationwide but offers no forecasting capability. NASA FIRMS [11] provides near-real-time global fire detection data from MODIS and VIIRS satellite instruments, enabling quantification of fire activity as an explanatory variable for PM2.5 episodes. Open-Meteo [12] provides free retrospective and forecast meteorological data suitable for academic applications.

No published operational system to date combines these three data sources into an automated daily forecasting and alerting pipeline for Chiang Mai. The primary novelty of CAAS lies in integrating multi-source data acquisition, feature engineering, ML model training and comparison, automated deployment, and continuous monitoring within a single production-grade pipeline aligned with current MLOps best practices.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Overview

The ChiangMai Air Quality Alert System (CAAS) is designed as a fully automated end-to-end MLOps pipeline. The system ingests data from three sources daily, generates PM2.5 regression forecasts and binary alert classifications for the next 1, 3, and 7 days, and triggers automated health alerts when concentrations are projected to exceed the 50 µg/m^3 threshold. A monitoring subsystem continuously evaluates forecast accuracy and detects data drift, triggering automated retraining when degradation is detected.

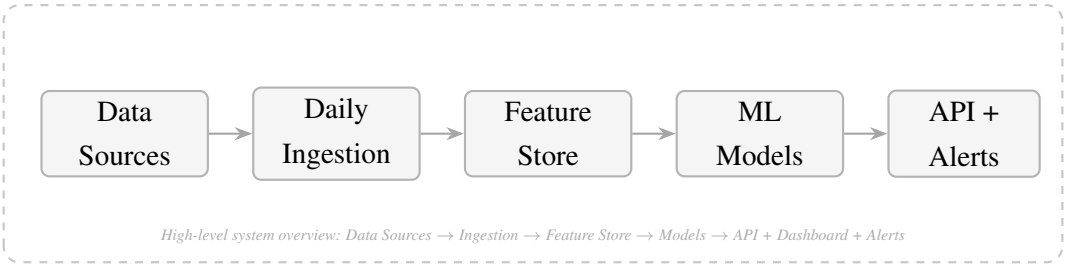

Figure 3.1: High-level overview of the ChiangMai Air Quality Alert System (CAAS).

3.2 Data Pipeline Design

3.2.1 Data Sources

The pipeline integrates three data sources, each providing a distinct signal relevant to PM2.5 forecasting in Northern Thailand.

Table 3.1: Summary of integrated data sources

Source	Provider	Type	Coverage & Granularity
PM2.5 concentration	air4thai / PCD Thailand	Excel + REST API	2011–2025, stations 35T & 36T, Chiang Mai (daily average)
Historical weather	Open-Meteo API	REST API	Chiang Mai coordinates (daily)
Fire hotspots	NASA FIRMS API	REST API	Northern Thailand bounding box (daily)

The primary station is 35T (Chiang Mai City Hall, Chang Phueak), with station 36T

(Yupparaj School, Si Phum) serving as a fallback during gaps. The two stations exhibit a Pearson correlation of $r = 0.94$, confirming 36T as a reliable substitute for short-duration absences at the primary station. The full training dataset comprises approximately 5,475 daily observations spanning 15 years.

3.2.2 Ingestion Method

Data ingestion follows a two-mode strategy. A one-time historical load parses all 15 PCD Excel files (one per year, 2011–2025) to build the initial training corpus. Ongoing ingestion runs daily at 06:00 local time via a scheduled GitHub Actions workflow, retrieving the previous day's data from all three sources before executing the forecast pipeline.

Batch ingestion (rather than streaming) is appropriate for this use case because daily forecast granularity provides no operational benefit from sub-hourly data updates.

3.2.3 Transformation Pipeline

Transformations are applied in the following order within each pipeline execution:

1. **Schema validation** — verify expected columns, data types, and valid date range; pipeline halts immediately on any violation with no silent failures.
2. **Missing value handling** — interpolate PM2.5 gaps of up to three consecutive days using lag and rolling mean; substitute 36T station data for longer gaps; set fire count to zero on days with no detected hotspots.
3. **Unit normalisation** — standardise column names, numeric types, and date formats across all three source schemas.
4. **Feature engineering** — compute all derived features as described in Section 3.3.
5. **Target variable creation** — generate PM2.5 forward-shifted targets for $t + 1$, $t + 3$, and $t + 7$.

Orchestration is managed by GitHub Actions (cron: `0 6 * * *`). Each step logs success or failure explicitly, and checkpoints are saved after each step to enable pipeline resumption without re-running from scratch.

3.3 Feature Engineering

The full feature set comprises 44 engineered features organised into two groups: temporal autocorrelation features and contextual environmental features.

3.3.1 Temporal Features

Table 3.2: Temporal autocorrelation and calendar features

Feature	Description
`PM25_lag1/2/3/7/14`	PM2.5 values 1–14 days prior; captures short-term autocorrelation
`PM25_roll3/7/14/30_mean`	Rolling mean over 3–30 day windows; captures medium-term trend
`PM25_roll7/30_std`	Rolling standard deviation; quantifies pollution volatility
`month_sin`, `month_cos`	Cyclical month encoding; avoids ordinality assumption
`day_sin`, `day_cos`	Cyclical day-of-year encoding
`is_haze_season`	Binary: 1 if January–April
`is_rainy_season`	Binary: 1 if May–October
`is_cool_season`	Binary: 1 if November–December

3.3.2 Contextual Environmental Features

Table 3.3: Contextual meteorological and fire activity features

Feature	Description
`north_avg_PM25`	Average PM2.5 across 15 Northern Thailand stations; regional smoke signal
`north_max_PM25`	Maximum PM2.5 across Northern Thailand stations
`wind_speed`, `wind_dir`	Wind speed and direction from Open-Meteo; governs smoke transport
`humidity`, `temperature`	Atmospheric conditions from Open-Meteo
`boundary_layer_height`	Atmospheric mixing height; key driver of surface PM2.5 accumulation
`fire_count`	Daily hotspot count within 100 km of Chiang Mai (NASA FIRMS)
`fire_frp_sum`	Total Fire Radiative Power; proxy for fire intensity and smoke volume

3.3.3 Target Variables

Table 3.4: Target variables for regression and classification tasks

Variable	Type	Definition
`PM25_next1`	Regression	PM2.5 concentration tomorrow (μg/m^3)
`PM25_next3`	Regression	PM2.5 concentration in 3 days
`PM25_next7`	Regression	PM2.5 concentration in 7 days
`alert_next1`	Binary classification	1 if PM2.5 tomorrow $>$ 50 μg/m^3
`alert_next3`	Binary classification	1 if PM2.5 in 3 days $>$ 50 μg/m^3

3.4 Model Training and Evaluation

3.4.1 Model Selection Rationale

This project employs a two-model comparative strategy rather than assuming *a priori* that one architecture will outperform the other. This decision is motivated by an important consideration raised during proposal review: with approximately 5,475 daily

observations, does a sequential deep learning model offer genuine advantages over a well-tuned gradient boosting model with rich temporal features?

The honest answer is: not necessarily. The comparative experimental design allows the data to answer this question empirically.

XGBoost — Primary Baseline Model

XGBoost [4] is the primary baseline and a strong candidate for the production model. Its key advantages for this dataset include:

- Efficient and robust training on moderate-sized tabular data with limited overfitting risk.
- Native handling of missing values without imputation.
- Strong regularisation (L1/L2) to prevent overfitting on the approximately 5,475-sample training set.
- Interpretability via SHAP feature importance analysis, enabling model behaviour explanation to non-technical stakeholders.
- Given that the temporal features already encode autocorrelation and rolling patterns explicitly, XGBoost operates on a feature representation that has effectively "pre-digested" the sequence structure into a flat tabular form—which is precisely where gradient boosting excels.

LSTM — Comparative Sequential Model

An LSTM network [5] with a 30-day input window is deployed as a comparative sequential model. The rationale for including LSTM is not an expectation of across-the-board superiority, but a specific testable hypothesis:

Hypothesis: XGBoost is expected to perform strongly on t+1 forecasting, where the lag-1 feature is highly predictive and the fixed-window feature representation is adequate. LSTM may demonstrate advantages on longer-horizon forecasts (t+3, t+7), where multi-step sequential dependencies and the compounding of seasonal atmospheric signals may be better captured through explicit sequence modelling than through the static lag features available to XGBoost.

Regarding the sample size: while the dataset contains approximately 5,475 row-level observations, training an LSTM with a 30-day sliding window generates approximately 5,400 overlapping sequence samples of length 30. Regularisation measures including dropout (rate 0.2–0.3), L2 weight decay, and early stopping will be applied to mitigate overfitting risk. If LSTM training proves unstable or overfitting cannot be controlled, the XGBoost model will serve as the sole production model—a fully acceptable out-

come since the comparative evaluation is itself the contribution.

3.4.2 Chronological Data Split

A strict chronological split is used with no data shuffling, respecting the temporal dependency structure and simulating realistic deployment conditions.

Table 3.5: Chronological training, validation, and test split

Split	**Period**	**Rationale**
Training	2011–2022	12 years; includes multiple complete haze seasons
Validation	2023	Hyperparameter tuning; one complete annual cycle
Test	2024–2025	Fully unseen; includes the most recent haze seasons

CHAPTER 4

SYSTEM ARCHITECTURE AND PIPELINE DESIGN

4.1 Architecture Overview

The CAAS architecture follows a layered design with clearly separated concerns across ingestion, storage, compute, serving, and monitoring layers. All components are deployed on AWS using the available Learner Lab credits.

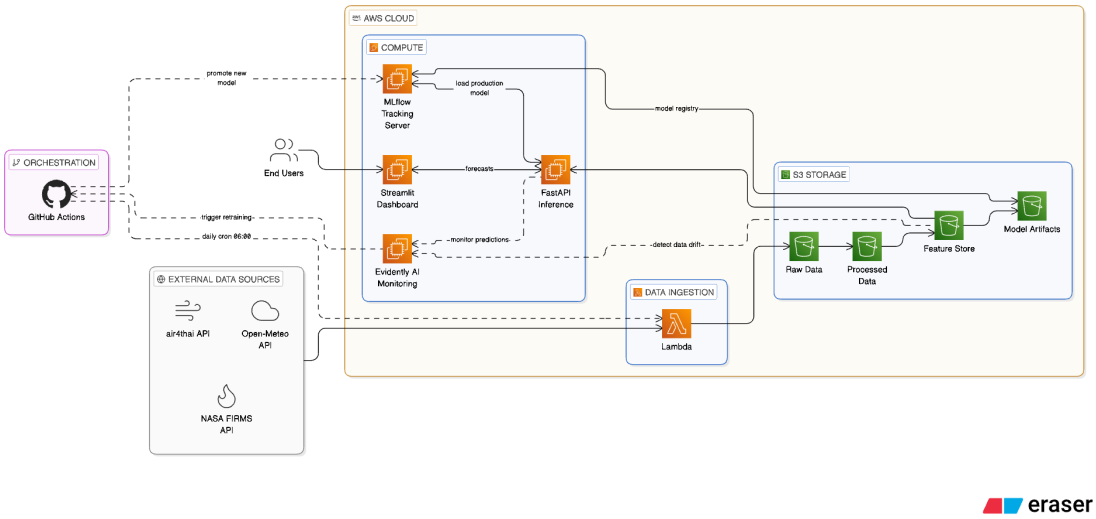

Figure 4.1: AWS cloud architecture for the ChiangMai Air Quality Alert System (CAAS). The pipeline spans three external data sources (air4thai API, Open-Meteo API, NASA FIRMS API), an AWS Lambda ingestion layer triggered daily at 06:00 UTC+7 via GitHub Actions, four S3 storage tiers (Raw, Processed, Feature Store, Model Artifacts), and a compute layer comprising MLflow Tracking Server, FastAPI Inference Server, Streamlit Dashboard, and Evidently AI drift monitoring. Dashed arrows indicate orchestration triggers and automated retraining signals.

4.2 Storage Layers

The storage architecture separates data by processing stage into four layers on Amazon S3:

- **Raw Layer** (`s3://caas/raw/pm25/`, `raw/weather/`, `raw/fire/`) — Immutable, append-only source data exactly as ingested. Files are never overwritten.
- **Processed Layer** (`s3://caas/processed/merged/`) — Cleaned, validated, and schema-enforced daily merged files.

- **Feature Store** (`s3://caas/features/pm25_daily_chiangmai.csv`) — ML-ready dataset with all 44 features and target columns, versioned by run date.
- **Model Artifacts** — MLflow artifact store containing trained model files, hyperparameters, evaluation metrics, SHAP plots, confusion matrices, and loss curves per experiment run.

4.3 MLOps Lifecycle

4.3.1 Experiment Tracking

All training runs are logged to MLflow, recording: hyperparameters (learning rate, `n_estimators`, LSTM units, sequence length, dropout rate), evaluation metrics (RMSE, MAE, R^2, Precision, Recall, F1, AUROC), artifacts (model file, feature importance plot, confusion matrix, loss curves), and the dataset version (snapshot date) used for training. This enables full reproducibility and direct side-by-side comparison between XGBoost and LSTM runs.

4.3.2 Model Versioning and Promotion

Models are managed through the MLflow Model Registry using a three-stage promotion workflow:

$$[\text{Staging}] \longrightarrow [\text{Validation}] \longrightarrow [\text{Production}]$$

- **Staging** — Newly trained model awaiting metric evaluation.
- **Validation** — Model has passed RMSE and Recall thresholds; ready for deployment review.
- **Production** — Currently serving model; only one model occupies this stage at any time.

Promotion to Production is automated when metrics pass predefined thresholds. Previous Production models are archived, not deleted, to enable rollback.

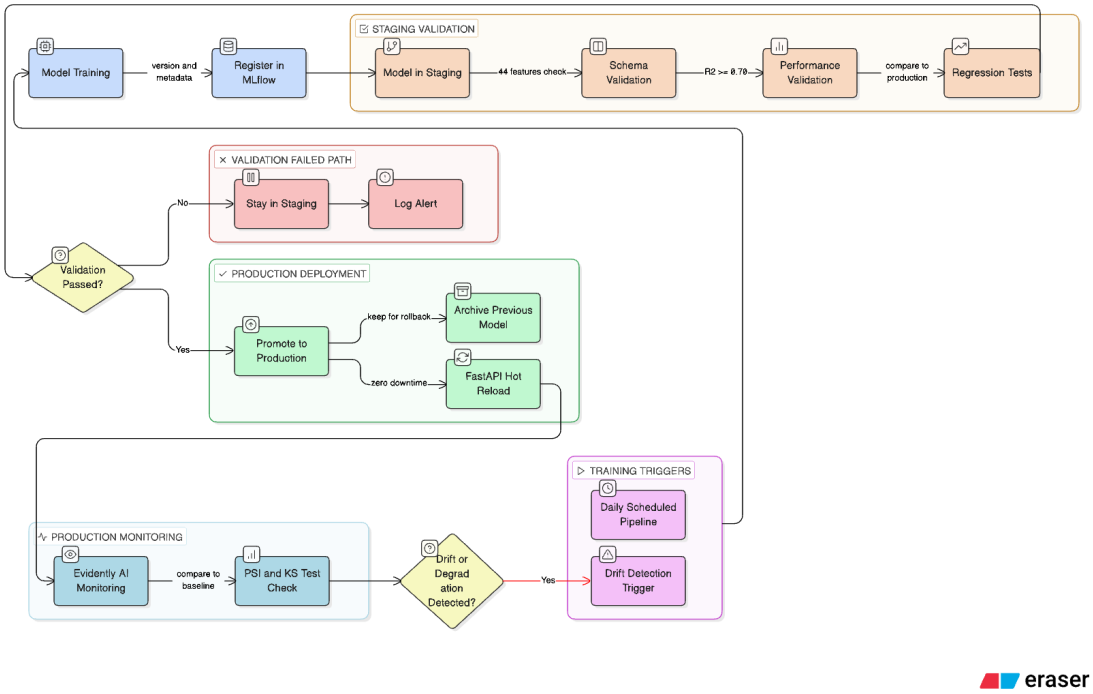

Figure 4.2: MLflow model promotion and lifecycle flow. Newly trained models are registered in MLflow and enter the Staging stage, where they undergo schema validation (44-feature check), performance validation ($R^2 \geq 0.70$), and regression tests against the current Production model. Failed validation keeps the model in Staging with an alert logged. On success, the model is promoted to Production with zero-downtime FastAPI hot-reload; the previous Production model is archived for rollback. Evidently AI drift detection and a daily scheduled pipeline serve as the primary retraining triggers.

4.3.3 Deployment Strategy

The deployment architecture uses a hybrid batch and real-time API approach. The primary daily pipeline runs at 06:00 via GitHub Actions: it ingests the previous day's data, generates t+1/3/7 forecasts, and stores results to S3. A FastAPI service exposes `/predict` and `/alert` endpoints for the Streamlit dashboard and external consumers.

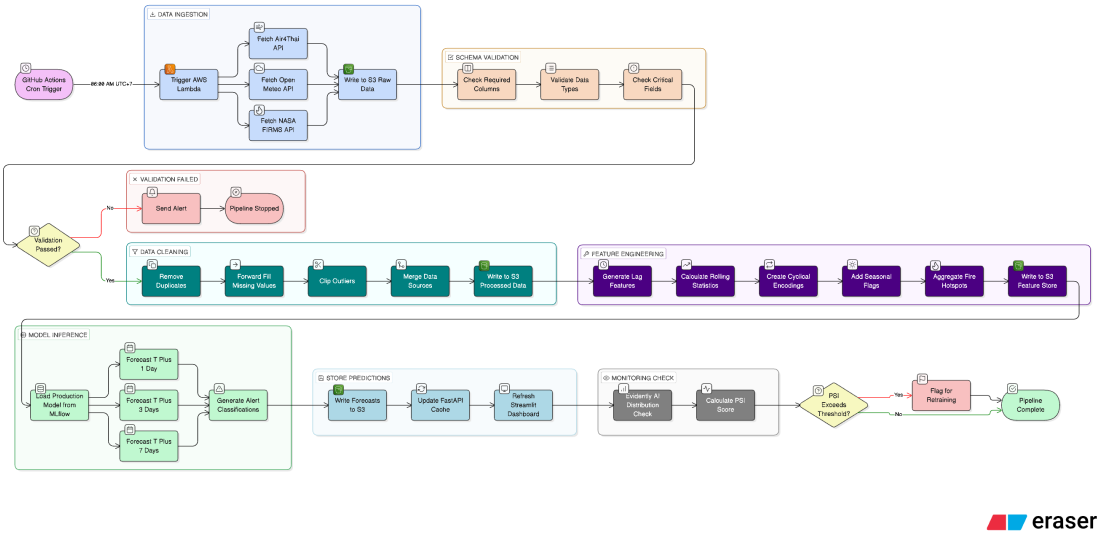

Figure 4.3: Daily MLOps pipeline execution flow triggered at 06:00 UTC+7 via GitHub Actions. The pipeline comprises seven stages: (1) Data Ingestion via AWS Lambda fetching from air4thai, Open-Meteo, and NASA FIRMS APIs into S3 Raw; (2) Schema Validation with a failure branch that sends an alert and stops the pipeline; (3) Data Cleaning including duplicate removal, forward-fill for missing values, and outlier clipping; (4) Feature Engineering producing 44 features (lag, rolling statistics, cyclical encodings, seasonal flags, fire hotspots) stored to S3 Feature Store; (5) Model Inference generating PM2.5 regression values and alert-level classifications for t+1, t+3, and t+7 horizons; (6) Store Predictions to S3 and update the FastAPI cache; (7) Monitoring Check using Evidently AI PSI drift detection, flagging for retraining if PSI > 0.2.

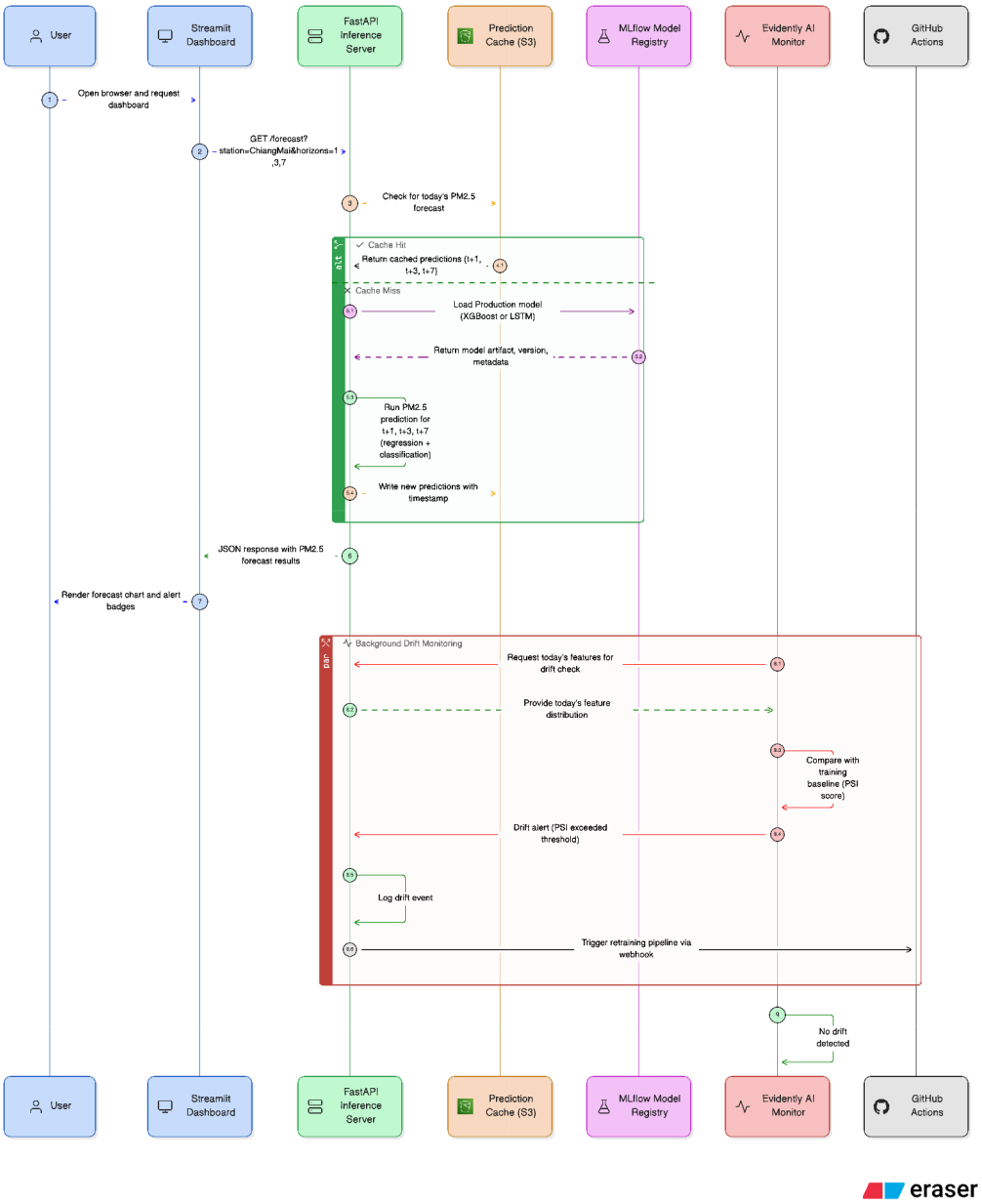

Figure 4.4: End-to-end PM2.5 forecast request flow for the CAAS system. When a user opens the Streamlit Dashboard, it sends a `GET /forecast?station=ChiangMai&horizons=1,3,7` request to the FastAPI Inference Server. The server first queries the S3 Prediction Cache: on a cache hit, predictions for t+1, t+3, and t+7 days are returned immediately; on a cache miss, the server loads the Production model from MLflow, runs PM2.5 regression and alert-level classification inference, writes the results to the cache, and returns a JSON response. In parallel, Evidently AI performs background drift monitoring by comparing today's feature distribution against the training baseline using PSI scoring. If PSI exceeds 0.2, a drift alert triggers the GitHub Actions retraining pipeline via webhook.

4.4 Monitoring and Automated Retraining

4.4.1 Data Drift Monitoring

Evidently AI generates weekly distribution comparison reports for key features (`PM25_lag1`, `north_avg_PM25`, `wind_speed`, `fire_count`) using Population Stability Index (PSI) and Kolmogorov-Smirnov tests against the training data baseline. Drift scores exceeding defined thresholds are flagged for team review before any automated retraining decision.

4.4.2 Model Performance Monitoring

After each daily batch run, the previous day's PM2.5 prediction is compared against the now-known actual observation. A 7-day rolling RMSE and MAE are logged to MLflow. Automated retraining is triggered when:

- Rolling RMSE exceeds 120% of the training baseline, or
- The 30-day alert false negative rate exceeds 15%.

Scheduled retraining also occurs in December each year, prior to the haze season, incorporating the most recent full year of data to reflect current environmental patterns.

4.5 Infrastructure and Cost Estimate

Table 4.1: Estimated monthly AWS cloud infrastructure cost

Service	**Usage**	**Est. Monthly Cost**
Amazon S3	1–2 GB data + model artifacts	\$0.03
AWS Lambda	Daily batch pipeline, ∼5 min/day	\$0.00 (free tier)
EC2 t3.micro	FastAPI serving + MLflow (always-on)	\$7–8 (or \$0 with Learner Lab credits)
Open-Meteo API	Daily weather fetch	\$0.00 (free)
NASA FIRMS API	Daily fire data fetch	\$0.00 (free)
Total		**\$2–8/month**

The primary architectural trade-offs considered were: Lambda vs. EC2 for serving (Lambda is cheaper but introduces cold-start latency; EC2 provides consistent response time for the dashboard—EC2 chosen for serving, Lambda for batch only); and self-hosted MLflow vs. managed service (self-hosted on EC2 avoids additional managed

service costs, acceptable at project scale).

4.6 Data Quality Management

Data quality is treated as a first-class concern in the pipeline. Each ingested record must pass schema validation before entering the processing layer; any violation halts the pipeline immediately with an explicit error log and GitHub Actions email notification. Silent failures are not permitted.

4.6.1 Missing Value Handling Strategy

Table 4.2: Missing value handling by data source

Source	Expected Missing Rate	Handling Strategy
air4thai PM2.5 (35T)	$\sim$2–3% per year	Gaps $\leq$3 consecutive days: linear interpolation using adjacent lag and rolling mean values. Gaps $>$3 days: substitute 36T station data ($r = 0.94$). All imputed rows flagged with `is_imputed = 1` column.
Open-Meteo weather	Near zero (reanalysis data)	No action required; reanalysis fills missing observations by design.
NASA FIRMS fire	Zero hotspots on some days is valid (not missing)	Fill `fire_count = 0` and `fire_frp_sum = 0` on days with no detected hotspots in the bounding box.

4.6.2 Schema Validation Rules

Every pipeline run enforces the following validation checks before processing proceeds:

- Expected column names and data types are present and correct.
- Date range is within bounds and contains no duplicate dates.
- Numeric values fall within physically plausible ranges (e.g., PM2.5 $\in [0, 500]$ µg/m^3).
- Station identifiers match expected codes (35T, 36T).

Earlier PCD files (2011–2017) may differ in column count due to network expansion; the validator applies source-year-specific schemas to handle this gracefully.

4.6.3 Pipeline Health Monitoring

Each pipeline step (ingest → validate → preprocess → feature engineering → predict) emits structured logs containing: step name, timestamp, input row count, output row count, and pass/fail status. If any step fails, all downstream steps are blocked immediately. A data freshness check after each ingestion run verifies that the latest PM2.5 record date matches yesterday; if data is more than two days stale, the source is flagged as potentially unavailable and an alert is raised.

CHAPTER 5

EXPERIMENTAL PLAN AND EXPECTED CONTRIBUTIONS

5.1 Dataset Characteristics

The training dataset comprises approximately 5,475 daily observations from 2011 to 2025, representing 15 complete annual cycles. Key statistical characteristics include:

- **March 2025 average**: 55.1 μg/m^3 — more than $3\times$ the WHO 24-hour guideline.
- **Single-day maximum**: 92.3 μg/m^3 (March 23, 2025).
- **Days exceeding Thailand's 25 μg/m^3 threshold**: 109 days (31% of the year).
- **PM2.5 missing rate**: approximately 2–3% per year.
- **Cross-station correlation** (35T vs. 36T): $r = 0.940$.

5.2 Baseline Methods for Comparison

Three strategies will be compared:

1. **Persistence baseline** — the forecast equals the most recent observed PM2.5 value (lag-1). This naive baseline establishes the lower bound that any model must exceed.
2. **XGBoost with 44 engineered features** — the primary tabular baseline described in Chapter 3.
3. **LSTM with 30-day sequence window** — the comparative sequential model.

5.3 Evaluation Metrics

Table 5.1: Evaluation metrics by task type and priority

Task	Metrics	Priority and Rationale
Regression (PM2.5 µg/m^3)	RMSE, MAE, R^2	RMSE is the primary regression metric; $R^2 \geq 0.70$ is the target benchmark.
Classification (alert/no alert)	Precision, Recall, F1, AUROC	Recall is the primary classification metric. A missed true alert (false negative) carries greater public health cost than a false alarm (false positive).

5.4 Experimental Setup

Experiments will be conducted as follows:

1. Both models trained on the 2011–2022 training set; hyperparameters tuned on the 2023 validation set using random search.
2. Final evaluation on the 2024–2025 test set (fully unseen data).
3. Separate experiments for each forecast horizon (t+1, t+3, t+7) to test whether any LSTM advantage is horizon-dependent.
4. XGBoost SHAP analysis conducted to identify the most influential features for interpretability and domain insight.
5. All runs logged to MLflow with full hyperparameter, metric, and artifact records for reproducibility.

5.5 Expected Contributions

This project is expected to deliver the following contributions:

1. **A production-ready MLOps pipeline for PM2.5 forecasting** demonstrating the full lifecycle from multi-source data ingestion to automated monitoring and retraining—applicable as a reference architecture for similar environmental mon-

itoring systems.

2. **An empirical comparison of XGBoost and LSTM** on a moderate-length daily environmental time series, providing evidence-based guidance on model selection for this class of problem.
3. **A public-facing health alert dashboard** providing actionable advance warning to Chiang Mai residents during the haze season.
4. **A reproducible multi-source feature engineering framework** integrating PM2.5 historical data, meteorological variables, and satellite fire detection into a unified ML-ready feature set.

5.6 Feasibility and Tool Selection

All tools are open-source, well-documented, and either free or within Learner Lab credit limits. The technology stack is:

- **Language**: Python 3.11
- **ML models**: XGBoost, TensorFlow/Keras (LSTM)
- **Experiment tracking**: MLflow
- **API serving**: FastAPI
- **Dashboard**: Streamlit
- **Drift monitoring**: Evidently AI
- **Orchestration / CI-CD**: GitHub Actions
- **Cloud**: AWS S3, Lambda, EC2 t3.micro (Learner Lab)
- **Data sources**: air4thai API (PCD), Open-Meteo API, NASA FIRMS API

The primary feasibility risk is LSTM training stability and overfitting given the dataset size. This risk is mitigated by the parallel XGBoost strategy: if LSTM proves problematic, XGBoost provides a fully capable production model.

5.7 Team Roles and Responsibilities

Table 5.2: Team member roles and task ownership

Member	Student ID	Responsibilities
Supanut Kompayak	st126055	Data pipeline design and implementation; feature engineering; model training (XGBoost + LSTM); experiment tracking (MLflow).
Shuvam Shrestha	st125975	FastAPI endpoint development; Streamlit dashboard; AWS cloud architecture design; cost estimation; CI/CD pipeline (GitHub Actions); Evidently AI monitoring setup; infrastructure deployment and maintenance.

5.8 Known Limitations and Risk Mitigation

Transparency about the project's limitations is essential for realistic feasibility assessment. Table 5.3 summarises the known risks and planned mitigations.

Table 5.3: Known limitations and risk mitigation strategies

Risk	**Description**	**Mitigation**
Dataset size for LSTM	5,475 daily observations is in the moderate-size regime where deep learning models are known to sometimes underperform well-tuned gradient boosting methods.	XGBoost serves as a fully capable fallback production model. Dropout and early stopping regularise LSTM. Results are reported honestly regardless of outcome.
air4thai API access	The air4thai API (`air4thai.com`) is blocked outside Thai IP ranges, limiting automated ingestion from cloud infrastructure hosted outside Thailand.	Historical data (2011–2025) acquired via direct Excel file download. For ongoing ingestion, a lightweight script runs on a Thai IP (e.g., AIT campus) or via VPN, then pushes to S3.
AWS Learner Lab expiry	Learner Lab credits may expire or reset during the semester, potentially disrupting always-on services.	All infrastructure is defined as code (GitHub Actions, configuration files) enabling rapid redeployment. MLflow and model artifacts are persisted to S3 to survive instance restarts.
Concept drift	Long-term shifts in haze season onset, duration, and intensity due to climate change may cause model performance to degrade between training and future operational use.	Annual scheduled retraining in December each year incorporates the most recent full year of data to reflect current environmental patterns.
Single-station dependency	If both 35T and 36T stations simultaneously experience extended outages, the pipeline has no PM2.5 source.	The data freshness check will raise an alert within 48 hours. Manual data acquisition from alternative PCD stations can be used as an emergency fallback.

5.9 Project Timeline

Table 5.4: Project milestones and target dates

Milestone	**Owner**	**Target Date**
Proposal submission	Both	March 12, 2026
Full dataset ready (all years + weather + fire)	Supanut Kompayak	March 15, 2026
XGBoost baseline trained and evaluated	Supanut Kompayak	March 15, 2026
LSTM model trained and compared	Supanut Kompayak	March 22, 2026
FastAPI + Streamlit dashboard deployed	Shuvam Shrestha	March 29, 2026
AWS infrastructure + CI/CD operational	Shuvam Shrestha	March 29, 2026
Monitoring and retraining pipeline live	Shuvam Shrestha	April 5, 2026
Final report and presentation	Both	TBD

REFERENCES

[1] World Health Organization (WHO). (2021). *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. World Health Organization. Geneva.

[2] Seinfeld, J. H., & Pandis, S. N. (2016). *Atmospheric Chemistry and Physics: From Air Pollution to Climate Change* (3rd ed.). John Wiley & Sons, New Jersey.

[3] Liang, Y., Ke, S., Zhang, J., Yi, X., & Zheng, Y. (2018). GeoMAN: Multi-level attention networks for geo-sensory time series prediction. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI-18)*, pp. 3428–3434. AAAI Press.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. ACM, New York.

[5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[6] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). The M4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4), 802–808.

[7] Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

[8] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 28. Curran Associates.

[9] Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Murching, A., Nykodym, T., Ogilvie, P., Parkhe, M., Singh, A., Xie, C., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45.

[10] Pollution Control Department, Thailand. (2024). *Air Quality and Noise Management: Air4Thai Monitoring System*. Retrieved from http://air4thai.com

[11] NASA EOSDIS. (2024). *Fire Information for Resource Management System (FIRMS)*. NASA Earthdata. Retrieved from https://firms.modaps.eosdis.nasa.gov

[12] Zippenfenig, P. (2023). Open-Meteo.com Weather API. *Zenodo*. https://doi.org/10.5281/zenodo.7970649